

ClipMind AI: Video Summarization & KeyMoments Detection

1. Project Name

ClipMind AI: Video Summarization & KeyMoments Detection: an AI-powered platform that converts lecture and video content into transcripts, summaries and key moments.

2. Milestones

Milestone 1: Project Initialization, Design Process and Core Setup

Milestone 1 established the working foundation: a FastAPI backend with secure registration, login with JWT, role-based access, validated video upload, upload history and status tracking on MongoDB Atlas, together with the Transcript and Summary APIs that carry the Whisper and AI summarization results to the database and the frontend.

Milestone 2: Key Moments Detection, Analytics, Testing and Deployment

Milestone 2 delivered the Key Moments, Highlights, Keywords and Analytics APIs, secured with JWT and returning consistent responses. Every endpoint was tested with Postman, and a Docker Compose setup was prepared as the base for deployment.

Milestone 3: Key Moments Detection and Analytics Dashboard

Milestone 3 made the key moments and analytics features work on real data. The defects that stopped real timestamps from reaching key moment detection were fixed, and the Usage Reports and Content Insights APIs were built and tested on real videos.

Milestone 4: Containerization and Deployment

Milestone 4 containerized the application with Docker and Docker Compose, using Caddy as the web server, and deployed the website on AWS. It is available at <https://clipmind.duckdns.org>.

3. Project Objective

The objective of the project is to build ClipMind AI as an AI-powered platform that converts lecture and video content into transcripts, summaries and key moments, so that a user can register, log in securely, upload a video, and view its transcript, summary, key moments, highlights, keywords and analytics.

The project was then validated as a real-data, end-to-end system, with no hardcoded or sample content in the path, and extended with the Evidence Lens feature and a deployment on AWS so that it can be opened and demonstrated online. My objective throughout was to design, build and verify the backend layer.

4. Project Description

ClipMind AI follows a layered architecture: User, then Frontend (React/Next.js), then FastAPI Backend, then Authentication and Role Management, then Database (MongoDB Atlas), then Video Upload and Storage, then FFmpeg Processing.

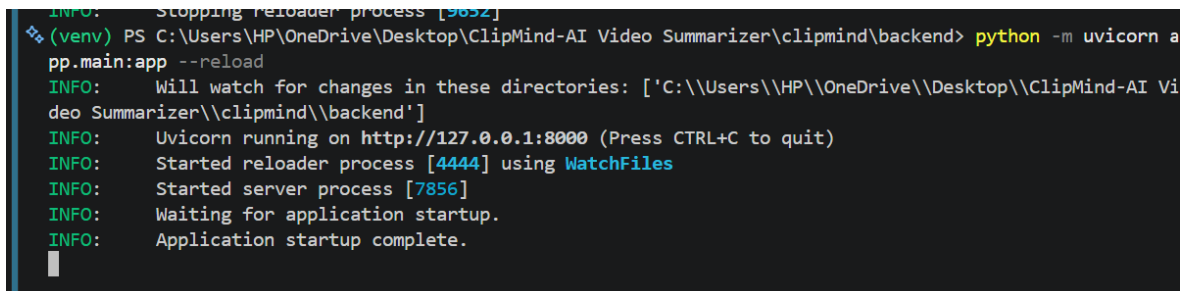
The processing flow is: Upload, FFmpeg audio extraction, Whisper transcription with timestamped segments, AI Summary, Key Moments, Highlights, Keywords, Analytics, Content Insights, Usage Reports, Evidence Lens, deployment on AWS. The results of each stage are stored with the video and returned through consistent JSON APIs protected by JWT authentication and role-based access control.

5. Technology Used

- Language: Python 3.13.7
- Backend Framework: FastAPI with Uvicorn
- Database: MongoDB Atlas, with Motor and Beanie
- Authentication and Security: JWT (python-jose), OAuth2 password form handling, bcrypt
- Validation: Pydantic, email-validator
- AI Pipeline: FFmpeg, OpenAI Whisper, Hugging Face Transformers (BART/T5)
- Analysis Methods: Deterministic content-based scoring and word-frequency analysis
- Containerization and Deployment: Docker, Docker Compose, Caddy, AWS
- API Testing: Swagger UI (/docs) and Postman
- Version Control: Git and GitHub
- Development Tool: Visual Studio Code

6. Screenshots

Figure 1: FastAPI server running (terminal showing "Application startup complete")

A terminal window with a dark background and light blue text. The prompt is '(venv) PS C:\Users\HP\OneDrive\Desktop\ClipMind-AI Video Summarizer\clipmind\backend>'. The command 'python -m uvicorn app:app --reload' has been executed. The output shows several INFO messages: 'Stopping reloader process [9632]', 'Will watch for changes in these directories: [\'C:\\Users\\HP\\OneDrive\\Desktop\\ClipMind-AI Video Summarizer\\clipmind\\backend\']', 'Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)', 'Started reloader process [4444] using WatchFiles', 'Started server process [7856]', 'Waiting for application startup.', and 'Application startup complete.'.

```
(venv) PS C:\Users\HP\OneDrive\Desktop\ClipMind-AI Video Summarizer\clipmind\backend> python -m uvicorn app:app --reload
INFO:      Stopping reloader process [9632]
INFO:      Will watch for changes in these directories: [\'C:\\Users\\HP\\OneDrive\\Desktop\\ClipMind-AI Video Summarizer\\clipmind\\backend\']
INFO:      Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:      Started reloader process [4444] using WatchFiles
INFO:      Started server process [7856]
INFO:      Waiting for application startup.
INFO:      Application startup complete.
```

Figure 2: Swagger UI (<http://127.0.0.1:8000/docs> showing Auth, Users and Videos routers)

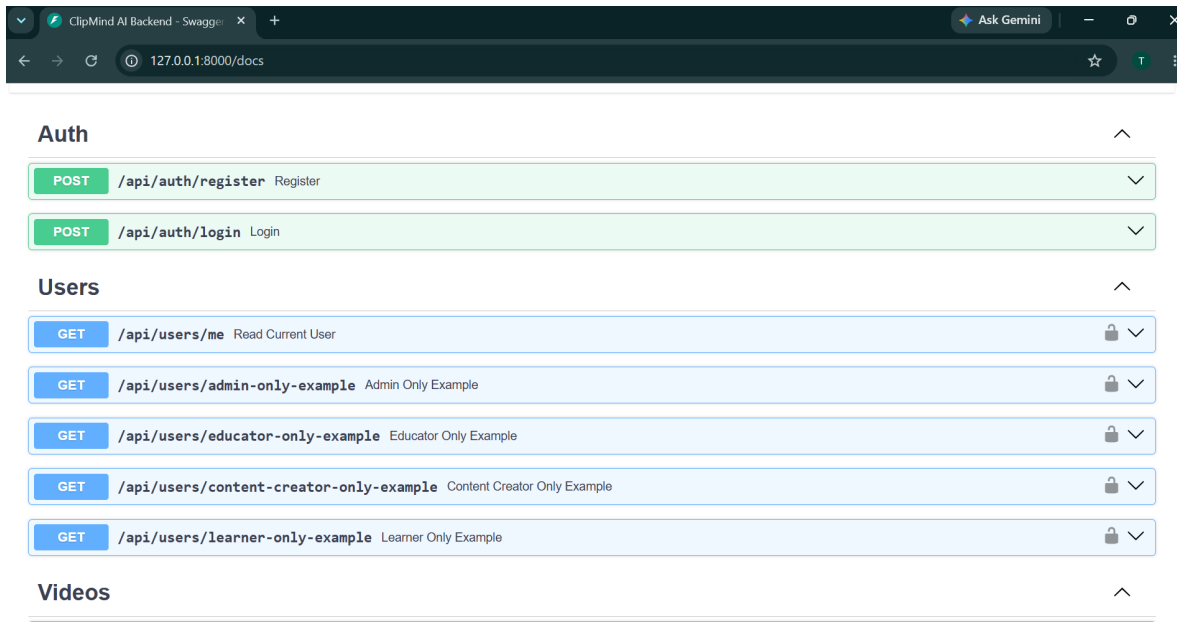


Figure 3: Successful registration (HTTP 201 response)

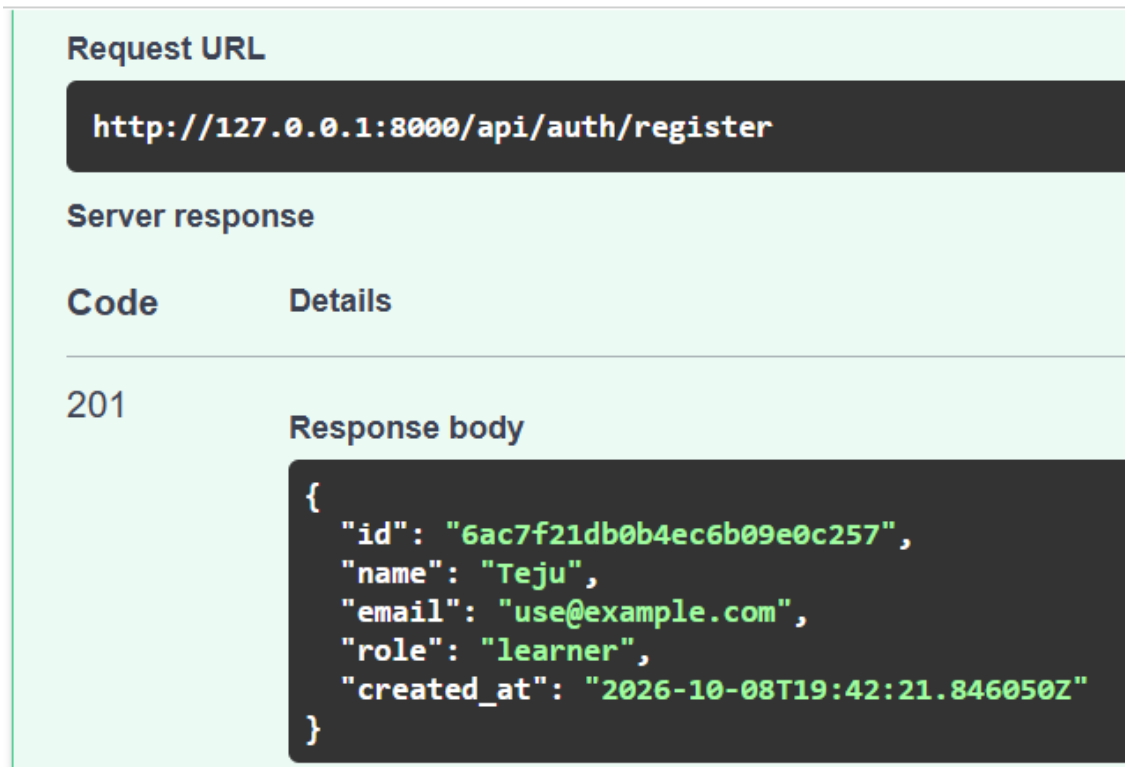


Figure 4: Successful login (HTTP 200 with JWT access token)

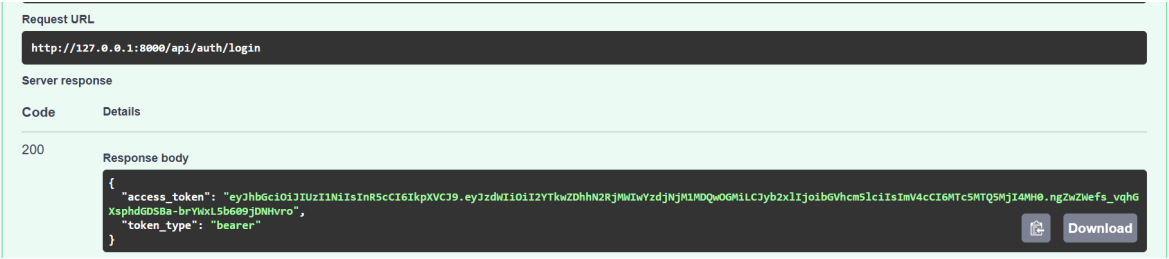


Figure 5: Key Moments API response (timestamps, topics and importance scores)

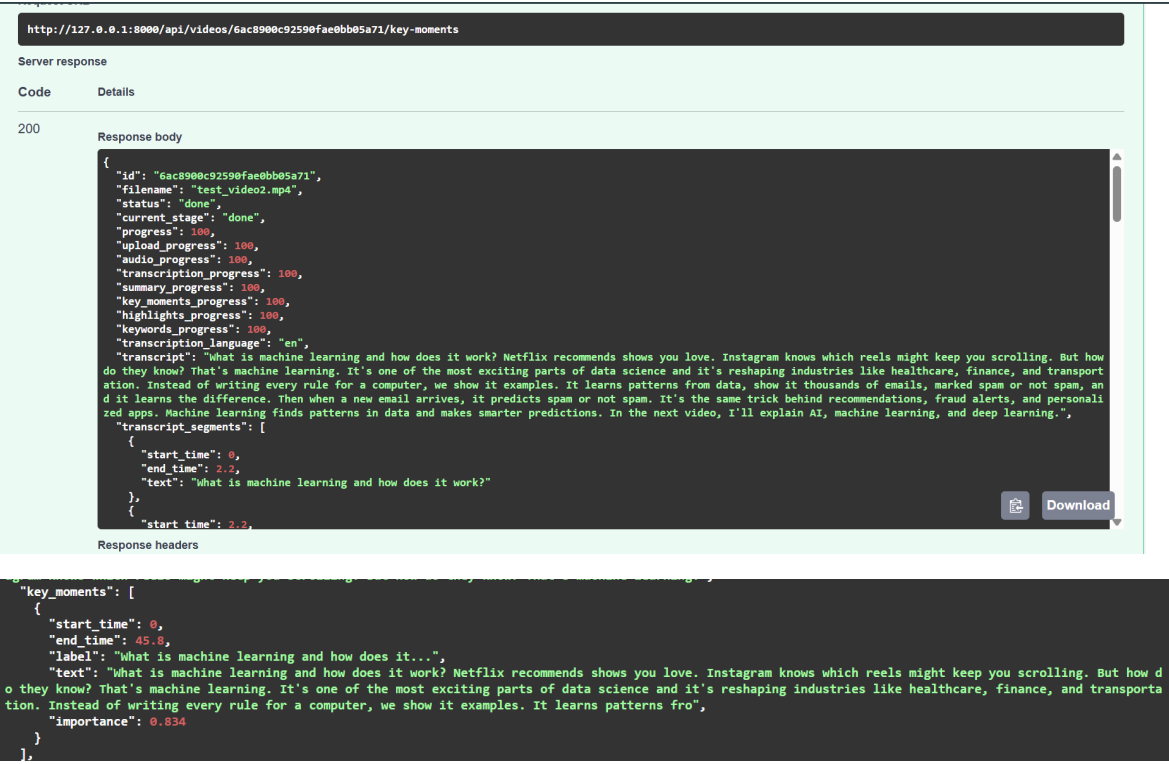


Figure 6: Highlights and Keywords API responses

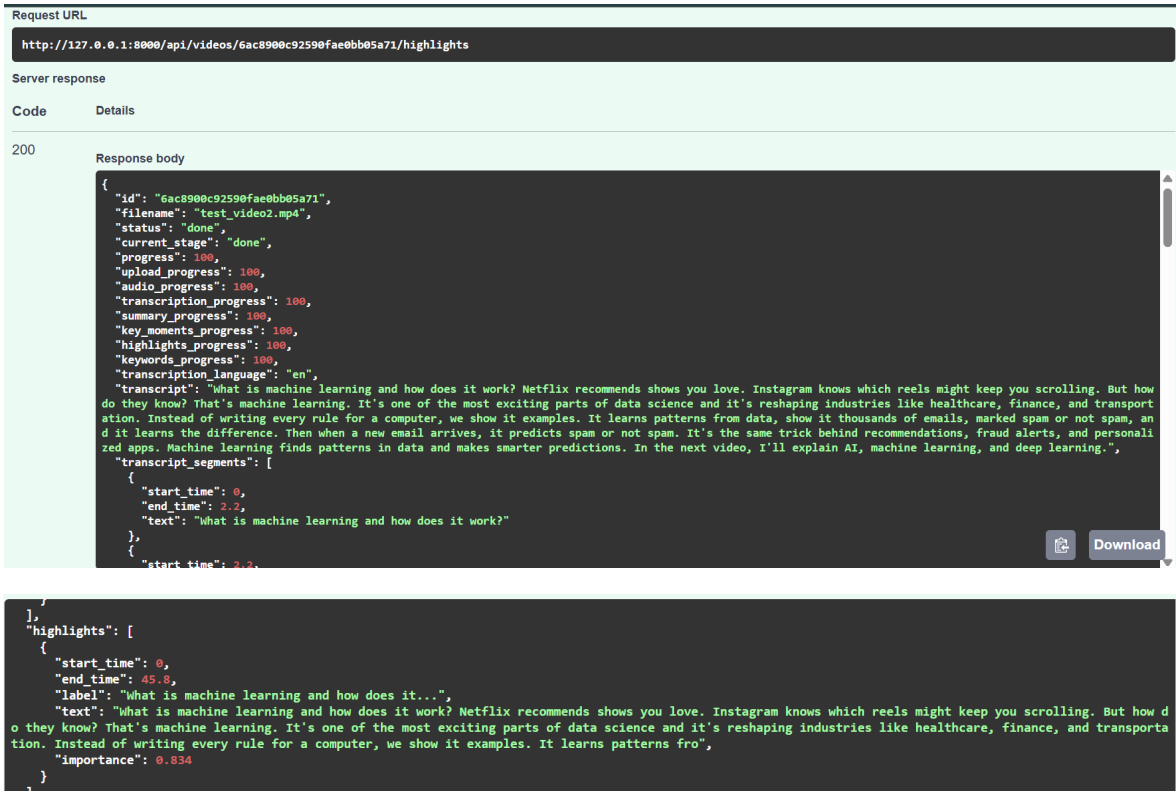


Figure 7: Analytics API response (word counts, key moment, highlight and keyword counts, and top keywords for a video)

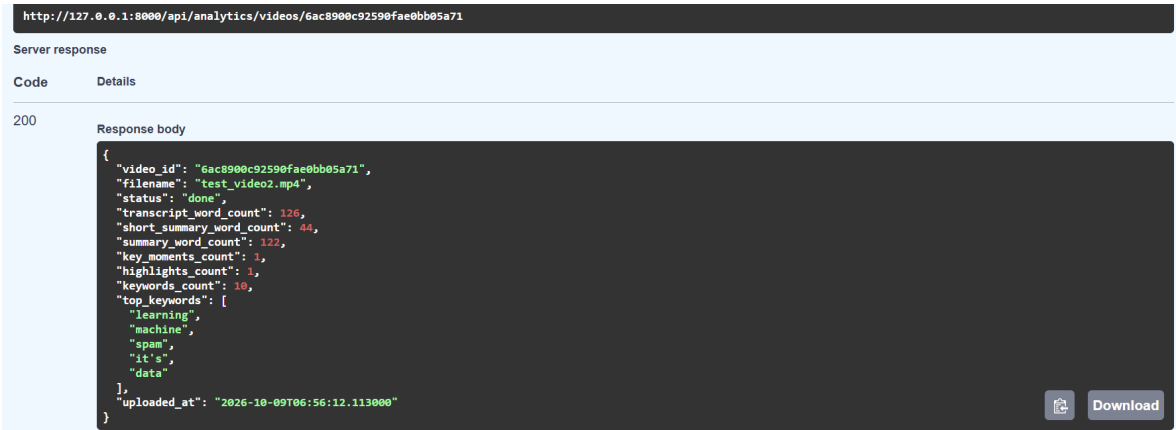


Figure 8: Transcript API response (transcript text with real timestamped segments)

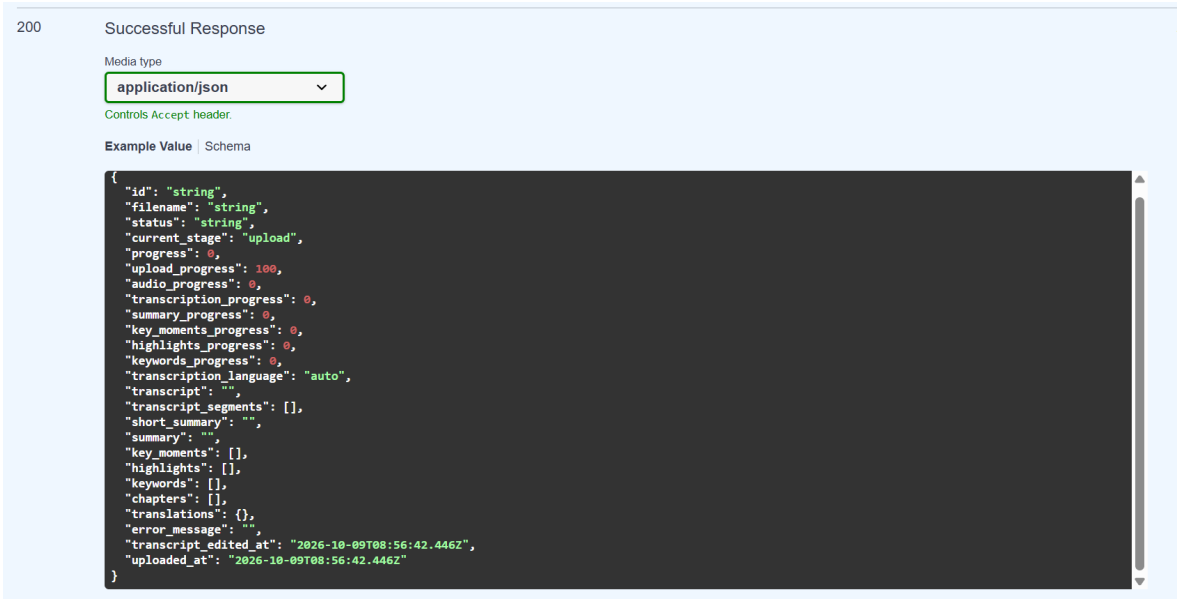


Figure 9: Usage Report API response (uploads by day, success and failure counts, storage)

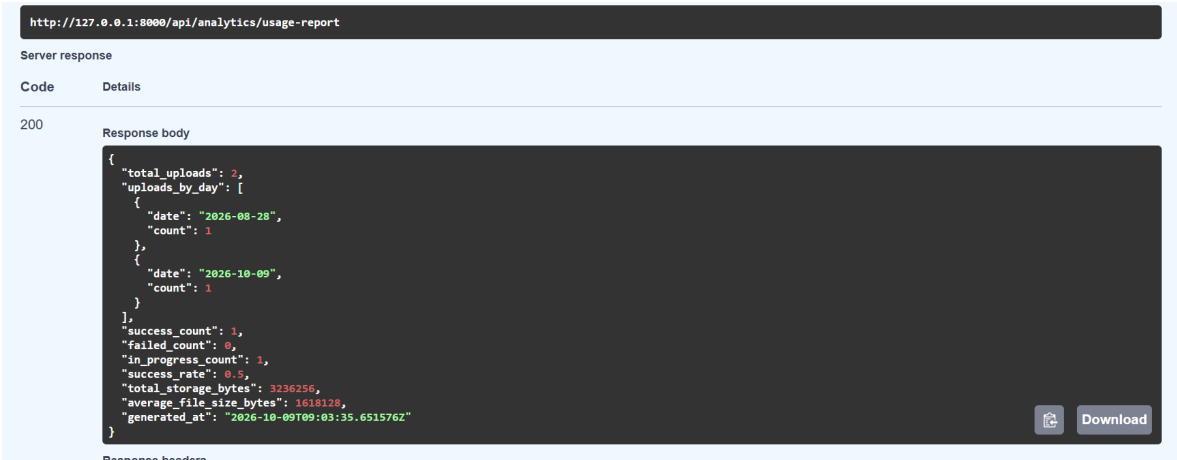
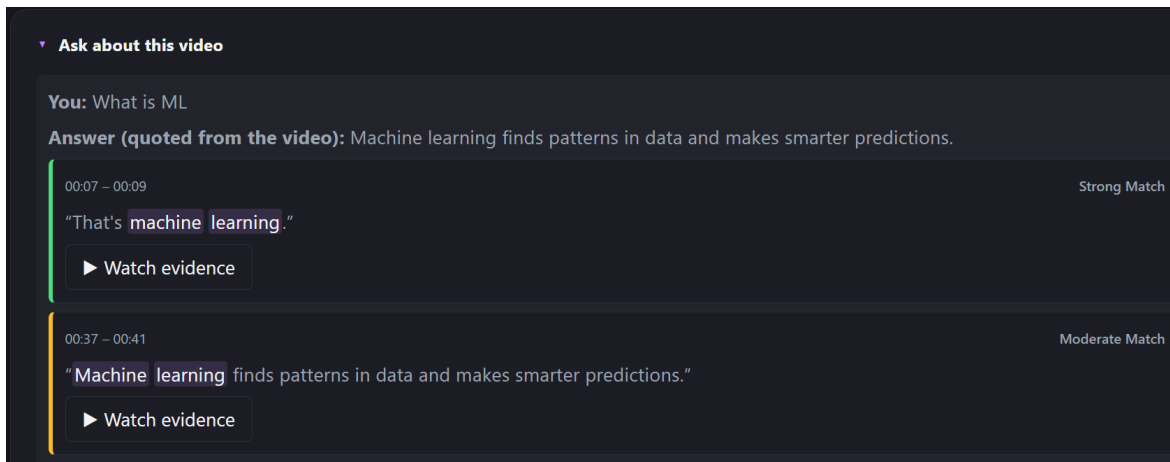


Figure 10: Evidence Lens (answer quoted from the video, with the supporting transcript evidence, its timestamp and the Watch evidence option)



7. Team Contribution

ClipMind AI was built by a team of four: Abhigna (key moments and integration), Amrita (AI/ML validation), Harika (frontend and analytics) and me (backend APIs and deployment). Each person's work depended on someone else's, so collaboration was part of every milestone.

My APIs depended on the key-moment output being stable, so I designed the API structure first and connected it to the real output as it became stable. I validated the schemas against real data so the frontend and the other members could rely on consistent response fields. I worked on a personal branch of the shared GitHub repository, and when work had to be combined in a short window, daily merges through Pull Requests kept the code in a working state.

This showed me how much a backend depends on clear agreements about response formats, and how much smoother integration becomes when changes are merged often and problems found in testing are fixed quickly.

8. Challenges Faced

- Database choice undecided. The plan required a team decision between PostgreSQL and MongoDB. I used SQLite temporarily, then migrated to MongoDB Atlas and rewrote the models, schemas and routes with Motor and Beanie once the team decided.
- Gaps against plan requirements. The first build lacked password strength checks, expired-token handling, RBAC for all four roles and file-size validation. I ran a gap analysis, added each missing item and verified it with live tests.
- Pull Request could not be opened. The shared repo had no main branch and the branches had unrelated histories. I raised it for team coordination.
- Dependency on upstream output. The Key Moments API could not be finalized until the key-moment logic produced real output. I designed the API structure first and connected it as it became stable.
- Inconsistent field names. I validated the schemas against real data and made the response fields consistent.
- Login form-data handling. The login endpoint needed to accept OAuth2 form-data and return an access token. I fixed the handling and re-verified the full authentication flow.
- Analytics numbers not matching the data. I verified the computed values against manually counted results and fixed the inconsistencies.
- Real timestamps were being lost. Whisper's segment timestamps were discarded, so key moments fell into fixed 30-second blocks. I changed transcription to return the segments and saved them to the video record.
- Open limitation. Whisper segments have no gap between them, so the 1.5-second merge threshold collapses most videos into one or two key moments. I recommend a maximum moment duration or merging by sentence or topic boundary.
- Learner role could not upload files. Uploading is controlled by role-based access control, so uploads are done from a content creator or educator account, and learners use the processed results.

- Large AI models on a cloud server. Whisper and Hugging Face models take a lot of storage and time to download. Keeping them in named volumes avoids downloading them again after a restart or redeploy.

9. Conclusion

ClipMind AI was developed over four milestones, from a secure backend foundation to a live, deployed platform. The backend now supports secure registration and login, validated video upload, transcripts and summaries, key moments, highlights, keywords, analytics, usage reports, content insights and Evidence Lens, verified with live requests including invalid input, unsupported formats, empty files and authentication failures.

The platform runs as a live website on AWS at <https://clipmind.duckdns.org>, so the whole flow from upload to analytics can be demonstrated online. One algorithm-tuning item, the merge behavior of key moments, remains open with a clear recommendation for follow-up.

10. Acknowledgement

I would like to express my sincere gratitude to Infosys Springboard for providing the Virtual Internship 7.0 and the opportunity to work on the ClipMind AI project. I thank my mentors for their guidance and feedback throughout the internship.

I also thank my teammates, Abhigna, Amrita and Harika, for their cooperation and support during every milestone. Working together on integration, testing and deployment made it possible to deliver the project end to end.